Java Class Execution in Java 1–21

① – Execution Lifecycle

Order	Phase	What Executes	Times
1	JVM Start	Bootstrap ClassLoader	JVM
2	Class Loading	".class" file loaded	Class
3	Verification	Bytecode verified	Class
4	Preparation	Static vars (default values	Class
5	Resolution	Symbolic refs —direct refs	Class
6	Initialization	Static vars + static blocks	Class
7	"main()	Starts the program	Class
8	Object Allocation		Method
9	Default Init	Instance vars (default).	Object
10	Explicit Init	Instance vars assigned	Object
11	Instance Blocks	{} blocks executed	Object
12	Constructor	Constructor body	Object
13	Object Ready	Object usable state	Object
14	GC Eligible	No references	Object
15	Memory Reclaim	Heap cleaned	JVM

③ – Java 1 → 21 → Similarities

- Static before non-static
- Parent before child.
- Instance blocks before constructor.
- Implicit 'super() first.
- Interface no instance lifecycle.
- GC non-determinstic.

***Core Java class** execution unchanged from **Java 1 to Java 21**, Features changed, but lifecycle did not.*

② – Static Execution Order (All Versions)

Static vars	Top → Bottom
Static blocks	Top → Bottom.
Static methods	When called.
Run before 'main()	Always

③ – Non-Static Execution Order (All Verssions)

Instance vars	JVM.
Instance vars	Top → Bottom.
Instance blocks	Top → Bottom.
Constructor	Last.
Run per object	Always.

④ – Java 1 → 21 → Similarities

- Static before non-static.
- Parent before child.
- Instance blocks before constructor.
- Implicit 'super() first.
- Interface no instance lifecycle.
- GC non-deterministic.

⑥ – Canonical Example for Java 8–21

```
Class LifeCycleDemo {
  static int x = initStatic();
  { .
    system our_println("Static Blocklock");
  }
  int x = initfestance();
  public static woid-mainojng(String() args)
    Conceee-println( "Madn-"statt");
    new LifeCycleDemo(); "Madn-tnd"""));
  }
```

Complete Lifecycle, Similarities, and Evolution of Java Class Execution from Java 1 to Java 21

Alright — this is **big-picture Java mastery** stuff 🧠🔥
I'll give you a **single, structured, interview-ready answer** that covers:

1. ✅ **Complete execution lifecycle** (static → non-static → GC)
2. ✅ **What executes when (step-by-step)**
3. ✅ **Similarities across Java 1 → 21**
4. ✅ **Dissimilarities / evolution across Java versions**
5. ✅ **One canonical example code**
6. ✅ **Everything in TABLE form** (as requested)

🔥 Java Class Execution Lifecycle (Java 1 → 21)

1 COMPLETE EXECUTION SEQUENCE (START → END)

📊 JVM → Class → Object → GC Lifecycle

Order	Phase	What Executes	Scope	Happens How Many Times
1	JVM Start	Bootstrap ClassLoader	JVM	Once
2	Class Loading	.class file loaded	Class	Once
3	Verification	Bytecode verified	Class	Once
4	Preparation	Static vars → default values	Class	Once
5	Resolution	Symbolic → direct refs	Class	Once
6	Initialization	Static vars + static blocks	Class	Once
7	main()	Program entry	Method	Once
8	Object Allocation	Memory allocated	Object	Per new
9	Default Init	Instance vars default	Object	Per new
10	Explicit Init	Instance vars assigned	Object	Per new
11	Instance Blocks	{} blocks	Object	Per new
12	Constructor	Constructor body	Object	Per new
13	Object Ready	Usable state	Object	Per new
14	GC Eligible	No references	Object	Optional

15	Memory Reclaim	Heap cleaned	JVM	Non-deterministic

2 STATIC vs NON-STATIC EXECUTION ORDER

📊 **Static Execution (All Java Versions)**

Step	Executes	Order Rule
1	Static variables	Top → Bottom
2	Static blocks	Top → Bottom
3	Static methods	On call only
4	Runs before main()	Always
5	Runs once per class	Always

📊 **Non-Static Execution (All Java Versions)**

Step	Executes	Order Rule
1	Instance vars (default)	JVM
2	Instance vars (explicit)	Top → Bottom
3	Instance blocks	Top → Bottom
4	Constructor	Last
5	Runs per object	Always

3 CLASS, ABSTRACT CLASS, INTERFACE – EXECUTION COMPARISON

📊 **Similarities & Differences (Execution Perspective)**

Feature	Class	Abstract Class	Interface
Static vars	✔	✔	✔ (constants)
Static blocks	✔	✔	✕

Instance vars	✔	✔	✕
Instance blocks	✔	✔	✕
Constructor	✔	✔	✕
Object creation	✔	✕	✕
Default methods	✕	✕	✔ (Java 8+)
Static methods	✔	✔	✔ (Java 8+)
Participates in instance lifecycle	✔	✔	✕

4 JAVA 1 → 21: WHAT NEVER CHANGED (SIMILARITIES)

📊 Core Rules That NEVER Changed

Rule	Java 1.0 → 21
Static executes before non-static	✔
Parent executes before child	✔
Instance blocks before constructor	✔
super() first in constructor	✔
Interface has no instance lifecycle	✔
GC is non-deterministic	✔
Class loading phases same	✔

5 JAVA 1 → 21: WHAT CHANGED (DISSIMILARITIES)

📊 Java Version Evolution Impacting Classes

Java Version	Change	Effect on Execution
Java 1.0	Basic class model	Core lifecycle defined

Java 1.2	Collections	No lifecycle change
Java 5	Generics, Annotations	Compile-time only
Java 6	Minor JVM tuning	No change
Java 7	Try-with-resources	Method-level only
Java 8	Default methods, static interface methods	Interface execution expanded
Java 9	Modules (JPMS)	Class loading visibility
Java 10	var	Compile-time only
Java 11	LTS	No lifecycle change
Java 14	Records (preview)	Constructor + fields auto
Java 16	Records final	Simplified object lifecycle
Java 17	LTS	Strong encapsulation
Java 19	Virtual threads (preview)	Thread lifecycle only
Java 21	Virtual threads stable	No class execution change

📌 **Important:**
👉 **Class execution order did NOT change from Java 1 → 21**
👉 Only **syntax, features, and memory optimizations** evolved

6 RECORDS (Java 16+) – SPECIAL CASE

📊 **Record Lifecycle vs Normal Class**

Aspect	**Class**	**Record**
Fields	Mutable	Final
Constructor	Manual	Auto-generated

Instance blocks	Allowed	✕
Lifecycle	Same JVM flow	Same JVM flow
Custom init	Yes	Limited

7 CANONICAL EXAMPLE (WORKS IN JAVA 8–21)

```
class LifeCycleDemo {

   static int a = initStatic();

   static {
      System.out.println("Static Block");
   }

   int x = initInstance();

   {
      System.out.println("Instance Block");
   }

   LifeCycleDemo() {
      System.out.println("Constructor");
   }

   static int initStatic() {
      System.out.println("Static Variable Initialized");
      return 10;
   }

   int initInstance() {
      System.out.println("Instance Variable Initialized");
      return 20;
   }

   public static void main(String[] args) {
      System.out.println("Main Start");
      new LifeCycleDemo();
      System.out.println("Main End");
   }
}
```

🔍 **Output (ALL Java versions)**

Static Variable Initialized
Static Block
Main Start
Instance Variable Initialized
Instance Block
Constructor
Main End

8 ONE MASTER INTERVIEW TABLE (CHEAT SHEET)

📊 **Ultimate Execution Order Summary**

Category	Order
Static	Parent → Child
Non-static	Parent → Child
Interface	Static only
Abstract class	Full lifecycle
Multiple interfaces	No instance lifecycle
Java 1 → 21	Same execution rules
GC	Non-deterministic

🎯 **PERFECT INTERVIEW ONE-LINER**

"From Java 1 to Java 21, the JVM class and object execution lifecycle has remained fundamentally the same; only language features evolved, not the execution order."

If you want next 🚀

- JVM memory diagram (Heap, Metaspace, Stack)
- Tricky output-prediction MCQs
- Interview rejection traps
- Java 21 virtual threads vs lifecycle

Java Class Execution Lifecycle in Java

Static Execution Order

Step	Stage	What Happens	Notes
1	**Load Class File**	✔.class loaded)	• JVM loads the class.
2	**Static Variables Init**	**(Defaults** (0 / null)	• Defaults (0 / null).
3	**Static Initializers**	(Static Blocks)	• Executed Top → Bottom.
4	**Static Methods**	Called	• When Invoked.

Non-Static Execution Order

Step	Stage	What Happens	Notes
1	**Memory Allocation**	Allocate Object.	• Allocate Object.
2	**Instance Variable Init**	Defaults → Explicit.	• Defaults → Explicit.
3	**Instance Initializer Blocks**	(Executed Top → Bottom)	• Executed Top → Bottom.
4	**Constructor Execution**	Constructor Runs.	• Constructor Runs.
5	**Object Ready**	Object Created.	

Garbage Collection

Step-by-Step Execution of Static and Non-Static Components of Java Classes

Great question — this is **core Java internals + interview gold** 🧠🔥
Below is a **complete, start-to-end execution sequence** of **static and non-static members**, **class loading**, **object lifecycle**, and **method execution** — explained **step by step** with **clear tables** and **one running example**.

📌 Java Class & Object Execution Lifecycle — Big Picture

Java execution happens in **two major lifecycles**:

1. **Class Lifecycle (Static context)** → happens **once per class**
2. **Object Lifecycle (Non-static context)** → happens **per object**

◆ 1. CLASS LOADING & STATIC EXECUTION SEQUENCE

☞ Happens only once, when the class is first referenced

📊 Static Execution Order Table

Step	Phase	What Happens	Notes
1	**Class Loading**	.class file loaded into JVM	By ClassLoader
2	**Linking – Verification**	Bytecode verification	Safety checks
3	**Linking – Preparation**	Static variables get **default values**	0, null, false
4	**Linking – Resolution**	Symbolic refs → memory refs	Optional
5	**Static Variable Initialization**	Explicit static values assigned	Top → Bottom
6	**Static Block Execution**	static {} blocks executed	Top → Bottom
7	**Class Ready**	Class ready for use	Static phase ends

📌 Important Static Rules

✅ Static variables & blocks run **in the order they appear**
✅ Static runs **before main()**
✅ Runs **only once**, even if multiple objects are created

□ Example: Static Execution

```
class StaticDemo {

   static int a = initA();

   static {
      System.out.println("Static Block 1");
   }

   static int b = initB();

   static {
      System.out.println("Static Block 2");
   }

   static int initA() {
      System.out.println("Static Variable A Initialized");
      return 10;
   }

   static int initB() {
      System.out.println("Static Variable B Initialized");
      return 20;
   }

   public static void main(String[] args) {
      System.out.println("Main Method");
   }
}
```

🔍 Output Order

```
Static Variable A Initialized
Static Block 1
Static Variable B Initialized
Static Block 2
Main Method
```

◆ 2. OBJECT CREATION & NON-STATIC EXECUTION SEQUENCE

☞ Happens every time new keyword is used

📊 Object Creation & Non-Static Order Table

Step	Phase	What Happens	Notes
1	**Memory Allocation**	Object memory allocated	Heap
2	**Default Initialization**	Instance variables → default values	0, null
3	**Instance Variable Initialization**	Explicit values assigned	Top → Bottom
4	**Instance Block Execution**	{} blocks executed	Top → Bottom
5	**Constructor Execution**	Constructor body runs	After blocks
6	**Object Ready**	Object usable	Lifecycle continues

📌 Important Non-Static Rules

✔ Instance variables & blocks execute **before constructor**
✔ Order = **Top to Bottom**
✔ Happens **for every object**

□ Example: Non-Static Execution

```
class InstanceDemo {

   int x = initX();

   {
      System.out.println("Instance Block 1");
   }

   int y = initY();

   {
      System.out.println("Instance Block 2");
   }

   InstanceDemo() {
```

```
        System.out.println("Constructor");
    }

    int initX() {
        System.out.println("Instance Variable X Initialized");
        return 5;
    }

    int initY() {
        System.out.println("Instance Variable Y Initialized");
        return 10;
    }

    public static void main(String[] args) {
        new InstanceDemo();
    }
}
```

🔍 Output Order

```
Instance Variable X Initialized
Instance Block 1
Instance Variable Y Initialized
Instance Block 2
Constructor
```

◆ 3. COMBINED STATIC + NON-STATIC EXECUTION (REAL INTERVIEW CASE)

📊 Full Execution Timeline Table

Order	Trigger	Execution
1	Class reference	Class loaded
2	Static vars	Default values
3	Static vars	Explicit initialization
4	Static blocks	Executed
5	main() starts	Entry point
6	new keyword	Object creation
7	Instance vars	Default values

8	Instance vars	Explicit initialization
9	Instance blocks	Executed
10	Constructor	Executed
11	Object ready	Use object

□ Full Example (Static + Non-Static)

```
class FullLifeCycle {

   static int s = initStatic();

   static {
      System.out.println("Static Block");
   }

   int i = initInstance();

   {
      System.out.println("Instance Block");
   }

   FullLifeCycle() {
      System.out.println("Constructor");
   }

   static int initStatic() {
      System.out.println("Static Variable Initialized");
      return 100;
   }

   int initInstance() {
      System.out.println("Instance Variable Initialized");
      return 200;
   }

   public static void main(String[] args) {
      System.out.println("Main Start");
      new FullLifeCycle();
      System.out.println("Main End");
   }
}
```

🔍 Final Output

Static Variable Initialized
Static Block
Main Start
Instance Variable Initialized
Instance Block
Constructor
Main End

◆ 4. OBJECT DESTRUCTION (GARBAGE COLLECTION)

📊 Object Destruction Lifecycle

Step	Phase	Description
1	Object unreachable	No references
2	GC eligible	JVM decides
3	finalize() (deprecated)	May run
4	Memory reclaimed	Heap cleanup

📌 **No guaranteed order or timing**

🔥 INTERVIEW ONE-LINER SUMMARY

"Static things execute once during class loading, top to bottom.
Non-static things execute per object creation, before constructor, top to bottom."

If you want next:

- 🔥 **Inheritance execution order (Parent → Child)**
- 🔥 **Static vs Non-static in abstract & interface**
- 🔥 **ClassLoader deep dive**
- 🔥 **Tricky MCQs & interview traps**

Inheritance Execution Order (Parent → Child) in Java

Static Execution Order

Step	Stage	Class	Notes
1	Static Variables Initialized (Defaults)	Parent	• JVM loads the class.
2	Static Variables Initialized (Explicit)	Parent	• Defaults (0 / null).
3	Static Initializers Blocks {}	Child	• Executed Top → Bottom.
4	Static Variables Initialized (Defaults)	Child	• Child, sreated Exxplict.
5	Static Initializer Blocks {}		

Runs Only Once Per Class

Non-Static Execution Order

Step	Stage	Class	Notes
1	Instance Variables	Parent	• Parent class loads.
2	Instance Variables Initialized (Explicit)	Parent	• Parent static vars & blocks execute.
3	Instance Initializer Blocks {}	Parent	• Child class loads.
4	Parent Constructor	Child	• Memory allocated.
5	Instance Variables Initialized (Defaults)		
6	Instance Initializer Blocks {}		
7	Child Constructor		
8	Object Ready		

Runs Every Time a Child Object is Created

Complete Execution Order

Order	Trigger	Execution
1	Class reference	Parent class loads.
2	Static Variables	Parent static vars & blocks execute.
3	Class reference	Child class loads.
4	Static variables	Child static vars & blocks execute.
5	new-Child()	Memory allocated.
6	Instance variables	Parent instance vars init.
7	Instance-blocks	Parent Instance blocks.
8	Constructor	Child constructor executes.
9	Constructor	Object ready for use.

Step-by-Step Sequence of Static and Non-Static Execution in Java Inheritance

Below is the **COMPLETE execution order of static + non-static members in Java inheritance (Parent → Child)**, from **JVM start to object ready**, explained **step-by-step in tables**, followed by a **single crystal-clear example**.

🔥 **Java Inheritance Execution Order (Parent → Child)**

🧠 **High-Level Rule (Memorize This)**

Static → Parent to Child (once)
Non-static → Parent to Child (every object)
Constructor → Parent to Child

🔹 **1. CLASS LOADING & STATIC EXECUTION (Inheritance)**

👉 **Happens once per class, first time it is referenced**

📊 **Static Execution Order — Parent → Child**

Order	Phase	Class	What Executes	Notes
1	Class Load	Parent	Static variables (default)	0, null
2	Class Init	Parent	Static variable initialization	Top → Bottom
3	Class Init	Parent	Static blocks	Top → Bottom
4	Class Load	Child	Static variables (default)	After parent
5	Class Init	Child	Static variable initialization	Top → Bottom
6	Class Init	Child	Static blocks	Top → Bottom
7	Ready	JVM	Classes ready	Static phase ends

📌 **Parent static always runs before Child static**

🔹 **2. OBJECT CREATION & NON-STATIC EXECUTION (Inheritance)**

☞ **Happens every time new Child() is called**

📊 **Object Creation Order — Parent → Child**

Order	Phase	Class	What Executes	Notes
1	Memory	Child	Object memory allocated	Heap
2	Init	Parent	Instance variables (default)	Before constructor
3	Init	Parent	Instance variable initialization	Top → Bottom
4	Init	Parent	Instance blocks	Top → Bottom
5	Constructor	Parent	Parent constructor	super()
6	Init	Child	Instance variables (default)	After parent
7	Init	Child	Instance variable initialization	Top → Bottom
8	Init	Child	Instance blocks	Top → Bottom
9	Constructor	Child	Child constructor	Last
10	Ready	JVM	Object ready	Use object

📌 **Parent constructor ALWAYS runs before Child constructor**

◆ **3. COMPLETE LIFECYCLE (STATIC + NON-STATIC + CONSTRUCTORS)**

📊 **Full Inheritance Execution Timeline**

Step	Trigger	Execution
1	Class reference	Parent class loads
2	Static init	Parent static vars & blocks

3	Class reference	Child class loads
4	Static init	Child static vars & blocks
5	main()	Starts execution
6	new Child()	Memory allocated
7	Parent init	Instance vars & blocks
8	Parent ctor	Parent constructor
9	Child init	Instance vars & blocks
10	Child ctor	Child constructor
11	Object ready	Object usable

◆ 4. REAL INTERVIEW-LEVEL EXAMPLE (🔥 MUST STUDY)

□ Code Example: Parent → Child Execution

```
class Parent {

   static int ps = initParentStatic();

   static {
      System.out.println("Parent Static Block");
   }

   int pi = initParentInstance();

   {
      System.out.println("Parent Instance Block");
   }

   Parent() {
      System.out.println("Parent Constructor");
   }

   static int initParentStatic() {
      System.out.println("Parent Static Variable Initialized");
      return 10;
   }
```

```
    int initParentInstance() {
        System.out.println("Parent Instance Variable Initialized");
        return 20;
    }
}

class Child extends Parent {

    static int cs = initChildStatic();

    static {
        System.out.println("Child Static Block");
    }

    int ci = initChildInstance();

    {
        System.out.println("Child Instance Block");
    }

    Child() {
        System.out.println("Child Constructor");
    }

    static int initChildStatic() {
        System.out.println("Child Static Variable Initialized");
        return 30;
    }

    int initChildInstance() {
        System.out.println("Child Instance Variable Initialized");
        return 40;
    }

    public static void main(String[] args) {
        System.out.println("Main Start");
        new Child();
        System.out.println("Main End");
    }
}
```

🔍 EXACT OUTPUT (100% Deterministic)

```
Parent Static Variable Initialized
Parent Static Block
Child Static Variable Initialized
Child Static Block
Main Start
Parent Instance Variable Initialized
```

Parent Instance Block
Parent Constructor
Child Instance Variable Initialized
Child Instance Block
Child Constructor
Main End

◆ 5. SUPER KEY INTERVIEW TRAPS 🚨

📊 Common Interview Traps Table

Trap	Reality
Child static runs first	✕ Never
Child constructor runs first	✕ Never
Instance blocks run after constructor	✕ Before constructor
Static runs per object	✕ Once per class
super() optional	✕ Always implicit

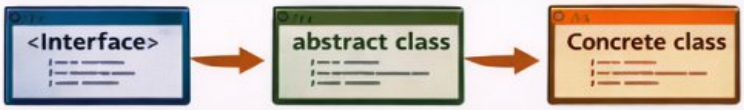
🎯 ONE-LINE INTERVIEW ANSWER

"In inheritance, Java always executes static members parent-to-child once, and non-static members parent-to-child per object before constructors."

If you want next 🔥 :

- super() vs this() execution order
- Abstract class + interface initialization order
- Multiple inheritance using interfaces
- Tricky MCQs with output prediction
- JVM memory diagram for inheritance

Static vs. Non-Static in Abstract & Interface Execution Order (Java)



Interface Static Execution Order

Step	Phase	Class	Notes
1	Interface Loaded	Parent	• JVM loads the class.
2	Static Constants Initialized	Parent	• Defaults (0 / null).
3	Static Methods Executed	Child	•

No Instance Lifecycle for Interfaces

Abstract Class Execution Order

Step	Phase	Class	Notes
1	Abstract Class Loaded	Parent	• Parent class loads.
2	Static Variables Initialized	Parent	• Abstract static var & bocks execute.
3	Static Blocks Executed	Child	• Child class loads.
4	Static Methods Executed	Child	• Memory allocated.

Runs Every Time a Child Object is Created

Complete Execution Order (Interface → Abstract → Concrete Class)

Order	Class	What Executes
1	Interface-A loaded	• Parent class loads:
2	Interface constants	• Interface constants Initialized.
3	Abstract loaded	• Abstract static vars &
4	Concrete loaded	• Concrete static vars & blocks executed.
5	new-Child()	• Memory allocated.
6	Abstract instance vars	• Abstract instance vars initialized.
7	Abstract instance vars	• Abstract instance blocks executed.
8	Constructor	• Abstract constructor executed.
9	new Concrete()	• Concrete instance vars initialized.
10	Abstract instance vars	• Abstract instance blocks executed.
11	Constructor	• Child constructor executes.

Step-by-Step Sequence of Static and Non-Static Execution in Abstract Classes & Interfaces

Below is a **clear, end-to-end execution order** for **static vs non-static in Interfaces and Abstract Classes**, shown **step by step, from JVM start to object ready**, with **tables + one solid example**.

🔥 **Static vs Non-Static Execution Order**

Abstract Class & Interface (Java)

🧠 **FIRST — THE GOLDEN RULES (Memorize)**

Rule	Reality
Interface has constructors	❌ Never
Interface instance variables	❌ All variables are public static final
Interface static blocks	❌ Not allowed
Interface static methods	✅ Java 8+
Interface default methods	✅ Java 8+ (non-static)
Abstract class static members	✅ Allowed
Abstract class non-static members	✅ Allowed

🔷 **1. INTERFACE EXECUTION LIFECYCLE (Static ONLY)**

👉 **Interface participates ONLY in static initialization**

📊 **Interface Static Execution Order**

Order	Phase	What Executes	Notes
1	Class Loading	Interface loaded	When referenced
2	Preparation	Constants get default values	0, null
3	Initialization	Interface variables initialized	Top → Bottom
4	Ready	Interface ready	No constructor

📌 **No instance lifecycle for interfaces**

Interface Example

```
interface MyInterface {

   int A = initA();

   static int initA() {
      System.out.println("Interface Variable A Initialized");
      return 10;
   }

   static void staticMethod() {
      System.out.println("Interface Static Method");
   }
}
```

◆ 2. ABSTRACT CLASS EXECUTION LIFECYCLE

👉 Behaves like a normal class, except cannot be instantiated

📊 Abstract Class Static Execution Order

Order	Phase	What Executes
1	Class Load	Abstract class loaded
2	Preparation	Static variables default
3	Initialization	Static vars initialized
4	Static Blocks	Executed top → bottom

📊 Abstract Class Non-Static Execution Order

Order	Phase	What Executes
1	Memory Allocation	Object memory
2	Default Init	Instance variables
3	Explicit Init	Instance variables
4	Instance Blocks	Executed
5	Constructor	Abstract class constructor

📌 **Abstract constructor executes during subclass creation**

◆ 3. COMBINED EXECUTION ORDER

Interface → Abstract Class → Concrete Class

📊 COMPLETE EXECUTION ORDER TABLE

Step	Trigger	What Executes
1	Class reference	Interface loaded
2	Static init	Interface constants
3	Class reference	Abstract class loaded
4	Static init	Abstract static vars & blocks
5	Class reference	Concrete class loaded
6	Static init	Concrete static vars & blocks
7	main()	Starts
8	new Concrete()	Object creation
9	Abstract instance vars	Init
10	Abstract instance blocks	Executed
11	Abstract constructor	Executed

12	Concrete instance vars	Init
13	Concrete instance blocks	Executed
14	Concrete constructor	Executed
15	Object ready	Usable

◆ 4. REAL INTERVIEW-LEVEL EXAMPLE

□ Code Example

```
interface I {

   int A = initA();

   static int initA() {
      System.out.println("Interface Variable A Initialized");
      return 1;
   }
}

abstract class AbstractBase implements I {

   static int B = initB();

   static {
      System.out.println("Abstract Static Block");
   }

   int x = initX();

   {
      System.out.println("Abstract Instance Block");
   }

   AbstractBase() {
      System.out.println("Abstract Constructor");
   }

   static int initB() {
      System.out.println("Abstract Static Variable Initialized");
      return 2;
   }
```

```
    int initX() {
        System.out.println("Abstract Instance Variable Initialized");
        return 3;
    }
}

class Concrete extends AbstractBase {

    static int C = initC();

    static {
        System.out.println("Concrete Static Block");
    }

    int y = initY();

    {
        System.out.println("Concrete Instance Block");
    }

    Concrete() {
        System.out.println("Concrete Constructor");
    }

    static int initC() {
        System.out.println("Concrete Static Variable Initialized");
        return 4;
    }

    int initY() {
        System.out.println("Concrete Instance Variable Initialized");
        return 5;
    }

    public static void main(String[] args) {
        System.out.println("Main Start");
        new Concrete();
        System.out.println("Main End");
    }
}
```

🔍 EXACT OUTPUT

```
Interface Variable A Initialized
Abstract Static Variable Initialized
Abstract Static Block
Concrete Static Variable Initialized
Concrete Static Block
Main Start
```

Abstract Instance Variable Initialized
Abstract Instance Block
Abstract Constructor
Concrete Instance Variable Initialized
Concrete Instance Block
Concrete Constructor
Main End

◆ 5. INTERVIEW TRAPS 🚨 (VERY IMPORTANT)

📊 Common Mistakes Table

Myth	Reality
Interface has instance lifecycle	✕
Interface static runs per object	✕ Once
Abstract constructor never runs	✕ Runs via subclass
Interface loads after class	✕ Loads first
Default methods run automatically	✕ Only when called

🎯 ONE-LINE INTERVIEW ANSWER

"Interfaces only participate in static initialization, while abstract classes follow full static and instance lifecycles just like normal classes during subclass creation."

If you want next 🔥 :

- Multiple interfaces + abstract class execution
- Diamond problem & default method resolution
- JVM memory diagrams for interface constants
- Output-based MCQs for interviews

Multiple Inheritance Using Interfaces
Execution Order in Java

Interface Static Execution Order

Order	Phase	Class	Notes
1	Interface-A	Interface-A Loaded	• JVM loads the class.
2	Interface-A	Static Constants Initialized	• Defaults (0 / null).
3	Interface-B	Interface-B Loaded	• Executed Top-• Bottom.
4	Interface-B	Static Methods Initialized	•

No Instance Lifecycle for Interfaces

Class Execution After Interfaces

Order	Phase	Class	Notes
1	Parent class (if any)	Static vars + blocks execute.	
2	Concrete class	Static vars + blocks execute.	
3	Class Ready	Class ready for use.	

Only Implementing Class Owns the Instance Lifecycle

Complete Execution Order (Interfaces → Class)

Order	Trigger		What Executes
1	Interface reference	Interface-**A loaded**	• Interface-A loaded.
2	Constants	Interface-A constants	• Interface-A constants initialized.
3	Interface reference	Interface-**B loaded**	• Interface-B loaded.
4	Constants	Interface-B constants	• Child static constants initialized.
5	Concrete reference	Concrete class loaded	• Concrete class static vars & blocks executed.
6	new Concrete()	Memory allocated.	• Memory allocated.
7	Instance vars	Instance vars	• Instance vars initialized.
8	Instance blocks	Instance blocks	• Instance blocks executed.
10	Constructor (Concrete)	Concrete constructor	• Concrete constructor executes.
11	Object ready	Object ready for use.	• Object ready for use.

Step-by-Step Sequence of Static and Non-Static Execution with Multiple Interfaces in Java

Below is the **complete lifecycle + execution order** for **multiple inheritance using interfaces**, shown **step-by-step in tables**, followed by **one precise example**.

🔥 Multiple Inheritance Using Interfaces – Java

Static & Non-Static Execution Order + Full Lifecycle

🧠 CORE TRUTHS (Foundation)

Concept	Reality
Multiple inheritance of state	✕ Not allowed
Multiple inheritance of behavior	✓ Via interfaces
Interface constructors	✕ Not allowed
Interface instance variables	✕ (all are public static final)
Interface instance blocks	✕
Interface static methods	✓
Interface default methods	✓ (non-static)
Interface participates in object creation	✕
Classes own the instance lifecycle	✓

◆ 1. INTERFACE LOADING & STATIC EXECUTION (Multiple Interfaces)

Happens **once**, when the interface is **first referenced**

📊 Interface Static Initialization Order

Step	Phase	Interface	What Executes	Notes
1	Load	Interface-1	.class loaded	JVM
2	Prepare	Interface-1	Constants default values	0 / null
3	Init	Interface-1	Constants initialized	Top → Bottom

4	Load	Interface-2	.class loaded	Independent
5	Prepare	Interface-2	Constants default values	
6	Init	Interface-2	Constants initialized	
7	Ready	JVM	Interfaces ready	No constructors

📌 **Order depends on first reference**, not implements order

◆ 2. CLASS LOADING AFTER INTERFACES

📊 Implementing Class Static Execution Order

Step	Phase	What Executes
1	Load	Class loaded
2	Prepare	Static variables default
3	Init	Static variables initialized
4	Init	Static blocks executed
5	Ready	Class ready

📌 **Interfaces initialize before the implementing class**

◆ 3. OBJECT CREATION & NON-STATIC EXECUTION

Happens **every time** new is used

📊 Object Creation Execution Order

Step	Phase	What Executes	Owner
1	Allocation	Memory allocated	Heap
2	Default Init	Instance variables	Class
3	Explicit Init	Instance variables	Top → Bottom
4	Instance Blocks	{} blocks	Top → Bottom
5	Constructor	Constructor body	Last
6	Ready	Object usable	JVM

📌 **Interfaces have NO instance lifecycle**

◈ 4. COMPLETE EXECUTION TIMELINE

Multiple Interfaces + Concrete Class

📊 FULL LIFECYCLE TABLE (Start → End)

Order	Trigger	Execution
1	Interface reference	Interface-A loaded
2	Static init	Interface-A constants
3	Interface reference	Interface-B loaded
4	Static init	Interface-B constants
5	Class reference	Implementing class loaded
6	Static init	Class static vars & blocks
7	main()	Execution starts
8	new keyword	Object memory allocated
9	Instance vars	Initialized

10	Instance blocks	Executed
11	Constructor	Executed
12	Method call	Default / overridden methods
13	End	Object in use

◆ 5. EXECUTION WITH DEFAULT METHOD CONFLICT

📊 Default Method Resolution Order

Priority	Winner
1	Class method
2	Superclass method
3	Explicit interface method (Interface.super)
✕	Ambiguous defaults (compile error)

◆ 6. COMPLETE EXAMPLE (INTERVIEW-GRADE)

□ Code Example

```
interface IA {
   int A = initA();

   static int initA() {
      System.out.println("IA static variable initialized");
      return 1;
   }

   default void show() {
      System.out.println("IA default show()");
   }
}

interface IB {
   int B = initB();
```

```
    static int initB() {
        System.out.println("IB static variable initialized");
        return 2;
    }

    default void show() {
        System.out.println("IB default show()");
    }
}

class MultiInheritanceDemo implements IA, IB {

    static int C = initC();

    static {
        System.out.println("Class static block");
    }

    int x = initX();

    {
        System.out.println("Instance block");
    }

    MultiInheritanceDemo() {
        System.out.println("Constructor");
    }

    static int initC() {
        System.out.println("Class static variable initialized");
        return 3;
    }

    int initX() {
        System.out.println("Instance variable initialized");
        return 4;
    }

    @Override
    public void show() {
        IA.super.show();
        IB.super.show();
        System.out.println("Class show()");
    }

    public static void main(String[] args) {
        System.out.println("Main Start");
        new MultiInheritanceDemo().show();
        System.out.println("Main End");
```

```
    }
}
```

🔍 EXACT OUTPUT ORDER

```
IA static variable initialized
IB static variable initialized
Class static variable initialized
Class static block
Main Start
Instance variable initialized
Instance block
Constructor
IA default show()
IB default show()
Class show()
Main End
```

🎯 INTERVIEW ONE-LINER (PERFECT ANSWER)

“In multiple inheritance using interfaces, only static initialization occurs for interfaces, while the implementing class alone controls object creation and non-static execution; default method conflicts must be explicitly resolved.”

If you want next 🔥

- Diamond problem (JVM-level explanation)
- Interface vs abstract class **memory diagram**
- Output-prediction MCQs
- Java 8 → Java 21 interface evolution

Diamond Problem JVM-Level Explanation (Java)

① What Is the Diamond Problem?

Aspect	Explanation
Problem Shape	A ← B, A ← C, B + C ← D
Core Issue	Ambiguous method resolution
Ambiguity	Which parent method, JVMT call?
Occurs in	Multiple inheritance of classes
JVM Concern	Method dispatch conflict
Java's Decision	Disallow multiple class inheritance

Key Rule: Two Single or Linherritance

② JVM-Level Reason

Hypothetical (Invalid Java)

	Class	Interface
A	void show()	B.show() or C.show()
B	overrides ()	overrides show()
C	overrides A)	overrides show()
D	extends B,C	illegal

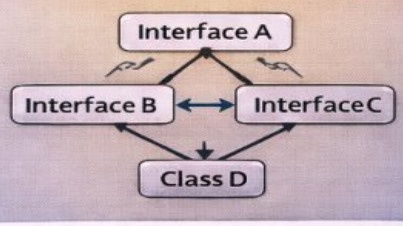
At JVM level: Two concrete superclasses → two vtables → conflict → undefined behavior.

⑤ Why Java Disallows Multiple Class Inheritance

JVM Aspect (Invalid Java)

1 Method Table	JVM uses vtables for dynamic dispatch
2 Conflict	Two parent classes define same method
3 JVM Limitation	No deterministic resolution rule
4 Memory Layout	Duplicate parent state
5 Result	Ambiguous method binding

⑤ Diamond with Interfaces (Java 8+ Default Methods)



Key Rule: Two concrete-superclasses → two vtables → conflict → undefined behavior.

④ Diamond with Interfaces (Java 8+)

Interface Diamond Strueture

State (fields)	Yes	No./(constants only)
VTM	oxcrettes	Default methods (optenal)
vttale	Concrete	Special resolution
JVM Anbiguity	High	Controlled

⑥ JVM Resolution Rules for Interface Diamond

Priority	Rule	
Class	method wins	▸ JVM binds class method
Interface B	Yes	▸ JVM binds parent
Class D	overrides	▸ JVM binds show()

⑦ Diamond with Interfaces (Java 8+)

> Interface A	default show()
> Interface B	default show()
> Class D	overrides show()
> JVM dispatch	Calls D.show()

⑲ JVM-Level Summary Table (One Glance)

Feature	Class Interfance	Interface
Multiple interfance	✗	✓ Allowed
Diamond problem	Prevented	✓ Controlled
JVM dispatch	High	✓ Eiminated
JVM ambiguity	Norte	Yes (at comiler)

"At the **JVM level, the diamond problem** arises from ambiguous vtable resolution; Java avoids it by **disallowing** multiple class inheritance and resolves it safely for interfaces via **explicit compile-time rules.**"

Absolutely — here is a **pure JVM-level, interview-grade explanation of the Diamond Problem in Java**, **strictly in table form**, covering **why it exists, why Java avoids it for classes, and how the JVM resolves it for interfaces (Java 8+)**.

🔥 Diamond Problem — JVM-Level Explanation (Java)

1 What Is the Diamond Problem?

📊 Concept Overview

Aspect	Explanation
Problem Shape	A ← B, A ← C, B + C ← D
Core Issue	Ambiguous method resolution
Ambiguity	Which parent method should JVM call?
Occurs In	Multiple inheritance of classes
JVM Concern	Method dispatch conflict
Java's Decision	Disallow multiple class inheritance

2 Why Java Disallows Multiple Class Inheritance

📊 JVM-Level Reason

JVM Aspect	Explanation
Method Table (vtable)	JVM uses vtables for dynamic dispatch
Conflict	Two parent classes define same method
JVM Limitation	No deterministic resolution rule
Memory Layout	Duplicate parent state
Result	Ambiguous method binding
Java Rule	Only **single class inheritance**

📌 **At JVM level:**

Two concrete superclasses → two vtables → conflict → undefined behavior.

---

3 Diamond Problem Example (Class Inheritance — NOT Allowed)

📊 **Hypothetical (Invalid Java)**

Class	Method
A	void show()
B extends A	overrides show()
C extends A	overrides show()
D extends B, C	✕ Illegal

JVM Question	Why it fails
Which show()?	B.show() or C.show()
JVM decision	Impossible
Compiler action	Compilation error

---

4 Why Interfaces Solve the Diamond Problem (Java 8+)

📊 **JVM-Level Difference (Interface vs Class)**

Aspect	Class	Interface
State (fields)	Yes	No (constants only)
Method implementation	Yes	Default methods (optional)
vtable	Concrete	Special resolution
Instance memory	Present	None

JVM ambiguity	High	Controlled

5 Diamond with Interfaces (Java 8+ Default Methods)

📊 Interface Diamond Structure

```
        Interface A
            |
  ------------------------
  |                      |
Interface B  Interface C
  |                      |
  ------- Class D -------
```

6 JVM Resolution Rules for Interface Diamond

📊 JVM Default Method Resolution Order

Priority	Rule
1	**Class method wins**
2	**Superclass method wins**
3	**Most specific interface wins**
4	**Explicit override required**
✖	Ambiguity without override → compile error

7 Diamond Problem — JVM Dispatch Table Behavior

📊 Method Resolution at Runtime

Case	JVM Action
Class overrides method	JVM binds class method
Parent class has method	JVM binds parent
Single interface default	JVM binds default
Multiple defaults	Compile-time error
Explicit override	JVM binds override

8 Why JVM Forces Explicit Override (Interfaces)

📊 JVM Safety Rule

JVM Concern	Solution
Ambiguous defaults	Force override
Runtime unpredictability	Compile-time resolution
No state duplication	Interfaces have no fields
Deterministic dispatch	Explicit developer choice

📌 **JVM does NOT guess** — it forces you to decide.

9 Example: Diamond with Interfaces (Resolved)

📊 Resolution Table

Component	Behavior
Interface A	default show()
Interface B	default show()
Class D	overrides show()
JVM dispatch	Calls D.show()

🔟 JVM-Level Summary Table (One-Glance)

Feature	Class Inheritance	Interface Inheritance
Multiple inheritance	✕ Not allowed	✓ Allowed
Diamond problem	✕ Prevented	✓ Controlled
JVM ambiguity	High	Eliminated
Resolution time	N/A	Compile-time

vtable conflict	Yes	No
Requires override	N/A	Yes (if conflict)

🎯 **PERFECT INTERVIEW ONE-LINER**

"At the JVM level, the diamond problem arises from ambiguous vtable resolution; Java avoids it by disallowing multiple class inheritance and resolves it safely for interfaces via explicit compile-time rules."

If you want next 🔥

- Diamond problem **bytecode-level**
- Interface default method **dispatch internals**
- C++ vs Java diamond comparison
- Output-based MCQs

Java Multithreading Execution Order & Lifecycle

① - Static Execution Order

Order	Phase	What Executes
1	JVM Start	Bootstrap ClassLoader
2	Class Load	".class" loaded
3	Preparation	Static vars (default values)
4	Initialization	Static vars initialized.
5	Static Blocks	·static { }
6	Class Ready	Class ready for objects

① Static Initialization Happens **ONCE**

② - Non-Static Execution Order

Order	Phase	What Executes
1	Memory Allocation	Object allocated
2	Default Init	Instance vars (0 / null)
3	Explicit Init	Instance vars assigned
4	Instance Blocks	{ }
5	Constructor	Constructor body
6	Object Ready	Object ready for use.

⑥ - Non-Static Happens Per Thread Object

NEW → RUNNABLE → RUNNING

(WAITING / TIMED_WAITING / BLOCKED) →
→ **RUNNABLE → TERMINATED**

③ - Non-Static Execution Order

NEW → <@111 2("') creates object

RUNNABLE → **start()** called / scheduled

RUNNING → CPU allocated / **run()** starts

WAITING → Infinite wait (optional)

BLOCKED → Lock unavailable.

TIMED WAITING → Wait(sleep(ms).

RUNNABLE → **run()** ends

③ - Complete Execution Order (Multithreading)

Order	Execution
1	JVM Start
2	Static class static init.
3	"main()" method starts.
4	For each "new Thread()
5	Constructor - Thread 2.
6	"**start()**" method call.
7	**run()**" method call.
8	THREAD STATES (BLOCKED, WAITING, etc.)
9	Thread terminates.
10	GC Eligible· Thread object Ceanup

Aspect	Static	Non-Static	Thread
Runs When	Class Loaded	Object Created	start()
Runs How Often	Once	Per Object	Per Thread
Memory Location	Method Area	Heap	Stack

Step-by-Step Sequence of Static, Non-Static, and Thread Execution in Java Multithreading

Below is a **clear, end-to-end execution sequence** for **Java multithreading**, covering:

- ✅ Static vs non-static execution
- ✅ Thread class lifecycle
- ✅ What runs once vs per thread
- ✅ JVM → Class → Object → Thread → Termination
- ✅ **Step-by-step tables**
- ✅ **One clean, interview-safe example**

🔥 Java Multithreading – Execution Order & Lifecycle

1 BIG PICTURE: WHAT EXECUTES & WHEN

Java multithreading has **three overlapping lifecycles**:

1. **Class lifecycle** (static – once)
2. **Object lifecycle** (non-static – per object)
3. **Thread lifecycle** (per thread)

◆ 2 CLASS LOADING & STATIC EXECUTION (THREAD CLASS)

👉 Happens **once**, even if 100 threads are created

📊 Static Execution Order

Step	Phase	What Executes	Runs How Many Times
1	JVM Start	Bootstrap ClassLoader	Once
2	Class Load	.class loaded	Once
3	Preparation	Static vars → default values	Once
4	Initialization	Static vars (explicit)	Once
5	Static Blocks	static {}	Once
6	Class Ready	Ready for threads	Once

📌 Static is NOT per thread

◆ 3 OBJECT CREATION (THREAD OBJECT)

👉 Happens **every time you create a thread object**

📊 Non-Static Execution Order (Thread Object)

Step	Phase	What Executes
1	Memory Allocation	Object allocated
2	Default Init	Instance vars (0 / null)
3	Explicit Init	Instance vars
4	Instance Blocks	{}
5	Constructor	Constructor body
6	Object Ready	Thread object created

📌 **Thread is NOT running yet**

◆ 4 THREAD LIFECYCLE (MOST IMPORTANT)

🧠 Thread States (Official JVM States)

NEW → RUNNABLE → RUNNING → (WAITING / TIMED_WAITING / BLOCKED)
→ RUNNABLE → TERMINATED

📊 Thread Lifecycle Step-by-Step

Step	State	Trigger	What Happens
1	NEW	new Thread()	Thread object created
2	RUNNABLE	start() called	JVM schedules thread
3	RUNNING	CPU allocated	run() executes
4	BLOCKED	Lock unavailable	Waiting for monitor
5	WAITING	wait() / join()	Waiting indefinitely

6	TIMED_WAITING	sleep() / join(ms)	Timed wait
7	RUNNABLE	Lock / time released	Back to queue
8	TERMINATED	run() ends	Thread dies

📌 **run() is never called directly by JVM unless start() is used**

◆ 5 COMPLETE EXECUTION SEQUENCE (START → END)

📊 Master Timeline (Multithreading)

Order	Trigger	What Executes
1	JVM start	JVM initialized
2	Class reference	Thread class loads
3	Static init	Static vars & blocks
4	main()	Main thread starts
5	new Thread()	Thread object created
6	Constructor	Thread constructor
7	start()	New call stack created
8	run()	Thread logic executes
9	Thread waits / blocks	Optional
10	run() ends	Thread terminates
11	GC eligible	Thread object cleanup

◆ 6 STATIC vs NON-STATIC vs THREAD EXECUTION

📊 Comparison Table

Aspect	Static	Non-Static	Thread
Runs when	Class load	Object creation	start()

Runs how often	Once	Per object	Per thread
Memory	Method Area	Heap	Stack
Affected by threads	✕	✕	✔
Can run parallel	✕	✕	✔

◆ 7 EXAMPLE 1: EXTENDING THREAD

```
class MyThread extends Thread {

   static {
      System.out.println("Static block executed");
   }

   {
      System.out.println("Instance block executed");
   }

   MyThread() {
      System.out.println("Constructor executed");
   }

   @Override
   public void run() {
      System.out.println("Thread running: " + Thread.currentThread().getName());
   }

   public static void main(String[] args) {
      System.out.println("Main start");

      MyThread t1 = new MyThread();
      t1.start();

      System.out.println("Main end");
   }
}
```

🔍 Output Order (One Possible Valid Order)

```
Static block executed
Main start
Instance block executed
Constructor executed
Main end
Thread running: Thread-0
```

📌 **Thread output order is NOT guaranteed**

◆ 8 EXAMPLE 2: IMPLEMENTING Runnable (BEST PRACTICE)

```
class MyTask implements Runnable {

    static {
        System.out.println("Static block");
    }

    {
        System.out.println("Instance block");
    }

    MyTask() {
        System.out.println("Constructor");
    }

    @Override
    public void run() {
        System.out.println("Running in: " + Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        System.out.println("Main start");

        Thread t = new Thread(new MyTask());
        t.start();

        System.out.println("Main end");
    }
}
```

◆ 9 COMMON MULTITHREADING TRAPS 🚨

📊 Interview Traps Table

Myth	Reality
run() creates new thread	❌
start() runs immediately	❌
Static runs per thread	❌

Thread order predictable	✕
sleep() releases lock	✕
wait() releases lock	✓

🎯 **PERFECT INTERVIEW ONE-LINER**

"In Java multithreading, static initialization happens once at class loading, object initialization happens per thread object creation, and actual parallel execution begins only after start() creates a new call stack for run()."

If you want next 🔥

- synchronized execution order
- Deadlock lifecycle diagram
- wait() vs sleep() vs join() table
- ThreadPoolExecutor lifecycle
- Virtual threads (Java 21) vs platform threads

Deadlock Lifecycle (Java – JVM Level)

① What Is a Deadlock?

Aspect	Explanation
Deadlock	Two or more threads permanently blocked
Cause	Each thread waits for a lock held by another
JVM State	Threads stuck in BLOCKED
Recovery	JVM does not resolve automatically
Detection	Manual / tooling

Key Rule: All four must be trie simultaneously

② Necessary Conditions for Deadlock (Coffman Conditions)

#	Condition	JVM-Level Meaning
1	Mutual Exclusion	Lock can be held by only one thread
2	Hold and Wait	Thread holds one lock, waits for another
3	No Preemption	JVM cannot force lock release
4	Circular Wait	Thread-A ⟶ Thread-B ⟶ Thread-A

All four must be true simultaneously

③ Deadlock Formation Lifecycle (Step-by-Step)

#	Step	Action	JVM State
1	T1	Acquires Lock-A	RUNNING
2	T2	Acquires Lock-B	RUNNING
3	T1	Requests Lock-B	BLOCKED
4	T2	Requests Lock-A	BLOCKED
5	JVM	No progress possible	DEADLOCK
	Threads	Never resume	BLOCKED forever

③ Deadlock Formation Lifecycle (Step-by-Step)

Step	Thread	JVM State	
1	Lock-A	–	Lock-B
2	Lock-B	–	Lock-A

④ JVM Thread State View During Deadlock

T1	Owns Lock-A	Waiting for
T2	Owns Lock-B	Waiting for

JVM thread state view auting deadlock

⑤ Deadlock Lifecycle Phases (High Level)

Thread Creation	Threads start execution
Lock Acquisition	Locks obtained
Lock Contention	Conflicting lock order
Circular Wait	Mutual waiting
Deadlock	Infinite block
Detection	Manual / Tool-based
Resolution	Code change / Restart

JVM detects but does not fix deadlocks

⑥ Deadlock Detection (JVM & Tools)

jstack	Dumps thread stack
JVM Thread Dump	Shows BLOCKED threads
JConsole	Visual deadlock detection
VisualVM	Lock graph analysis
ThreadMXBean	Programmatic detection

JVM detects but does not fix' **deadlocks.**

⑦ JVM Thread Dump Indicators

➤ BLOCKED	Waiting for monitor
➤ waiting to lock	Lock request
➤ locked	Lock held
➤ Circular reference	Deadlock confirmed

⑧ Deadlock Resolution (After It Happens)

Restart JVM	Releases all locks	JI(SCBBTECH
Kill thread	Unsafe (not recommended)	JEESS)
Code refactor	Permanent fix	Continuuas retries

⑩ Deadlock vs Starvation vs Livelock

Deadlock	Starvation		Starvation		Livelock
Threads.Blocked	YES ✗	None	YÉs	Some	Continuous retriies
Threads blocked	YES ✗	Some	Priority/ Lnfair-locks		Continuous retries

Deadlock Lifecycle (Java - JVM Level)

① What Is a Deadlock?

Aspect	Explanation
Deadlock	Two or more threads permanently blocked
Cause	Each thread waits for a lock held by another
JVM State	Threads stuck in **BLOCKED**
Recovery	JVM does NOT resolve automatcally
Detection	Manual / tooling

② Deadlock Formation Lifecycle (Step-by-Step)

Step	Owns Lock-A	Waiting For	JVM State
T1	Lock-A	Lock-B	RUNNING
T2	Lock-B	Lock-A	RUNNING
T1	Requests	Lock-B	BLOCKED
T2	Requests	Lock-A	BLOCKED
JVM	No progress possible		DEADLOCK

③ Deadlock Formation Lifecycle (Step-by-Step)

Step	Thread	Action	JVM State
1	T1	Acquires Lock-A	RUNNING
2	T2	Acquires Lock-B	RUNNING
3	T1	Requests Lock-B	BLOCKED
4	T2	Requests Lock-A	BLOCKED

④ JVM Thread State View During Deadlock

Thread	Owns Lock-A
T1	Waiting for Lock-B
T2	Waiting for Lock-A

JVM detects but does simultaneously

⑤ Deadlock Formation Lifecycle (Step-by-Step)

Step	Thread		JVM State
1	T1	Acquires Lock-A	RUNNING
2	T2	Acquires Lock-B	BLOCKED
5	JVM	No progress possible	DEADLOCK

All four must be true simultaneously

⑥ Deadlock Lifecycle Phases (High Level)

jstack	Dumps thread stack
JVM Thread	Shows BLOCKED threads
JConsole	Visual deadlock detection
VisualVM	Lock graph analysis
ThreadMXBean	Programmatic detection

⑤ Deadlock Detection (JVM & Tools)

Thread Creation	Threads start execution
Lock Acquisition	Locks obtained
Lock Contention	Conflicting lock order
Circular Wait	Mutual waiting
Deadlock	Infinite block
Detection	Manual / Tool-based
Resolution	Code change / Restart

⑦ JVM Thread Dump Indicators

Indicator	Waiting for monitor	Lock request
'BLOCKED'	Lock request	Lock held
'locked'	Circular reference	Deadlock confirmed

JVM detects but does not fix deadlocks.

⑧ Deadlock vs Starvation vs Livelock

⑩ Deadlock vs Starvation vs Livelock

Deadlock	Yes	✕ None	Cause
Starvation	Yes	✕ Some	Circular locks
Livelock	No	✕ None	Continuous retiies

"Deadlock occurs when threads enter a **circular wait for locks**; the JVM can detect it but cannot resolve it — only code changes or process restart can fix it."

Below is a **clear, JVM-level, interview-ready explanation of the Deadlock Lifecycle in Java**, presented **strictly in tables**, from **thread start → deadlock → detection → resolution**.

🔥 Deadlock Lifecycle (Java – JVM Level)

1 What Is a Deadlock?

📊 Definition Table

Aspect	Explanation
Deadlock	Two or more threads permanently blocked
Cause	Each thread waits for a lock held by another
JVM State	Threads stuck in BLOCKED
Recovery	JVM does **not** resolve automatically
Detection	Manual / tooling

2 Necessary Conditions for Deadlock (Coffman Conditions)

📊 JVM Conditions Table

#	Condition	JVM-Level Meaning
1	Mutual Exclusion	Lock can be held by only one thread
2	Hold and Wait	Thread holds one lock, waits for another
3	No Preemption	JVM cannot force lock release
4	Circular Wait	Thread-A → Thread-B → Thread-A

📌 **All four must be true simultaneously**

3 Deadlock Formation Lifecycle (Step-by-Step)

📊 Execution Timeline Table

Step	Thread	Action	JVM State
1	T1	Acquires Lock-A	RUNNING
2	T2	Acquires Lock-B	RUNNING
3	T1	Requests Lock-B	BLOCKED
4	T2	Requests Lock-A	BLOCKED
5	JVM	No progress possible	DEADLOCK
6	Threads	Never resume	BLOCKED forever

4 JVM Thread State View During Deadlock

📊 Thread States Table

Thread	Owns Lock	Waiting For	JVM State
T1	Lock-A	Lock-B	BLOCKED
T2	Lock-B	Lock-A	BLOCKED

5 Deadlock Lifecycle Phases (High Level)

📊 Lifecycle Phases Table

Phase	Description
Thread Creation	Threads start execution
Lock Acquisition	Locks obtained
Lock Contention	Conflicting lock order
Circular Wait	Mutual waiting
Deadlock	Infinite block

Detection	Manual / Tool-based
Resolution	Code change / Restart

6 Deadlock Detection (JVM & Tools)

📊 Detection Methods Table

Method	How It Works
jstack	Dumps thread stack
JVM Thread Dump	Shows BLOCKED threads
JConsole	Visual deadlock detection
VisualVM	Lock graph analysis
ThreadMXBean	Programmatic detection

📌 JVM **detects but does not fix** deadlocks.

7 JVM Thread Dump Indicators

📊 Thread Dump Symptoms Table

Indicator	Meaning
BLOCKED	Waiting for monitor
waiting to lock	Lock request
locked	Lock held
Circular reference	Deadlock confirmed

8 Deadlock Resolution (After It Happens)

📊 Resolution Table

Technique	JVM-Level Effect
Restart JVM	Releases all locks
Kill thread	Unsafe (not recommended)
Code refactor	Permanent fix
Reduce locking	Lower contention

9 Deadlock Prevention Strategies (Best Practices)

📊 Prevention Table

Strategy	JVM Impact
Lock ordering	Breaks circular wait
Use tryLock()	Avoids blocking
Timeout locks	Prevents infinite wait
Minimize sync blocks	Fewer locks
Use concurrent APIs	Lock-free internals

🔟 Deadlock vs Starvation vs Livelock

📊 Comparison Table

Issue	Threads Blocked	Progress	Cause
Deadlock	Yes	✕ None	Circular locks
Starvation	Yes	✕ Some	Priority / unfair locks
Livelock	No	✕ None	Continuous retries

🎯 **PERFECT INTERVIEW ONE-LINER**

“Deadlock occurs when threads enter a circular wait for locks; the JVM can detect it but cannot resolve it—only code changes or process restart can fix it.”

If you want next 🔥

- Deadlock **code example + thread dump**
- Livelock & starvation lifecycle
- Deadlock vs race condition
- JVM lock internals (monitorenter)

Static Variables, Methods, Blocks Executing BEFORE main() Starts

① Static Variables, Methods, Blocks Executing BEFORE 'main()

Runs **Once** Per Class During Class Loading

Static Item	What It Is Used For	When It Executes
Static Variable	Shared class-level data	During class initialization
Static Final Variable	Constants	During class initialization
Static Block	Complex, initialization logic De diver licad	During class initialization
Static Method		Only when called
Static Factory Method	Controlled obj ect creation	When called
Static Main Method	JVM entry	After static init Program start

Code Example: Static Executes BEFORE 'main()

Code Example: Static Executes BEFORE 'main()

```
class StaticExample {
  static int a = init();  // static variable
  static {        // static block
   System.out.println("Static block executed");
 }
  static int initA() {  // static method
   Sotic method ("Static variable initialized");
 }
  static void staticMethod()
   System.out.println("Static method called");
 }
 public static void main(String() args);
}
```

Output Order:

- Static variable initialized
- Static block executed
- Main started
- Static method called

② Non-Static Variables, Methods, Blocks Executing AFTER 'main()' Starts

Runs **Every Time** a New Object is Created

Instance Item	What It Is Used For	When It Executes
Instance Variable	Object-specific data	During object creation
Instance Block	Shared init logic	Before constructor
Constructor	Object intialization	After instance blocks
Instance Method	Object behavior	When called
Instance InParent CCiable	Inmesible object creation	During object Nemy object-stitt..
Non-Static Final Variable	Immutable object data	During object IDS, coring per object)
Non-Static in Multithreading	Paring object object	Child-object

Code Example: Non-Static Executes AFTER 'main()"

```
class NonStaticExample {
  int x = initx();    // instance variable
  {
    SyStem.out.println("Instance block executed");
  }
  NonStaticExample {  //  constructor
    System.out.println("I
      Systerxc.println("Constructor");
  }
  int initx() {  // instance method
    System.out.println(;
      Instance variable ininitialized");
  }
```

Output Order:

- 1 Main started
- 2 Instance variable initialized
- 3 Instance block executed
- 4 Gonstructor executed.
- 5 Instance method called

Complete Overview of Static and Non-Static Execution in Java Classes

Below is a **clear, interview-ready explanation** that separates **what runs BEFORE main() (static context)** and **what runs AFTER main() starts (non-static context) — strictly in table form**, with **concise code examples**.

◆ 1. STATIC: Executed BEFORE main() (Class Loading Phase)

Runs **once per class**, without creating any object

📊 Static Variable, Method, Block — Usage & Execution

Item	Used For	When It Executes	Key Points	Code Example
Static Variable	Shared class-level data	During class initialization (before main)	Top → Bottom order	static int x = init();
Static Block	Complex initialization logic	During class initialization	Executes once	static { ... }
Static Method	Utility / helper logic	Only when explicitly called	No object needed	static void util()
Static Constant	Global constants	During class loading	static final	static final int MAX = 10;
Static Factory Method	Controlled object creation	When invoked	Preferred over constructor	static MyClass of()
Configuration Loader	Load config / DB / cache	Before main()	One-time setup	Static block

✔ Static Code Example (Runs BEFORE main())

```
class StaticExample {

   static int a = initA();      // static variable

   static {
      System.out.println("Static block executed");
   }

   static int initA() {
      System.out.println("Static variable initialized");
      return 10;
   }
```

```
    static void staticMethod() {
        System.out.println("Static method called");
    }

    public static void main(String[] args) {
        System.out.println("Main started");
        staticMethod();
    }
}
```

🔍 Execution Order

Static variable initialized
Static block executed
Main started
Static method called

◆ 2. NON-STATIC: Executed AFTER main() Starts (Object Creation Phase)

Runs **per object**, requires new

📊 Non-Static Variable, Method, Block — Usage & Execution

Item	Used For	When It Executes	Key Points	Code Example
Instance Variable	Object-specific state	During object creation	Per object	int x = initX();
Instance Block	Common init logic	Before constructor	Top → Bottom	{ ... }
Constructor	Object initialization	After instance blocks	Mandatory	MyClass()
Non-static Method	Object behavior	When invoked	Needs object	obj.method()
Business Logic	Runtime processing	After object creation	Per request	Service logic

---

✅ Non-Static Code Example (Runs AFTER main() Starts)

```
class NonStaticExample {

   int x = initX();     // instance variable

   {
      System.out.println("Instance block executed");
   }

   NonStaticExample() {
      System.out.println("Constructor executed");
   }

   int initX() {
      System.out.println("Instance variable initialized");
      return 20;
   }

   void instanceMethod() {
      System.out.println("Instance method called");
   }

   public static void main(String[] args) {
      System.out.println("Main started");

      NonStaticExample obj = new NonStaticExample();
      obj.instanceMethod();
   }
}
```

🔍 Execution Order

```
Main started
Instance variable initialized
Instance block executed
Constructor executed
Instance method called
```

🔥 3. STATIC vs NON-STATIC — MASTER COMPARISON

Aspect	Static	Non-Static
Executes	Before main()	After main()
Trigger	Class loading	Object creation
Runs how often	Once	Per object
Requires object	✕	✔

Memory area	Method Area / Metaspace	Heap
Can access instance data	✕	✔
Typical use	Utility, config, constants	Business logic, state

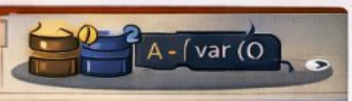
🎯 **INTERVIEW ONE-LINER (PERFECT ANSWER)**

“Static variables and blocks execute during class loading before main(), while non-static variables, blocks, and methods execute only after main() starts and an object is created.”

If you want next 👉
• Static vs non-static in **inheritance**
• Static in **abstract class & interface**
• Thread + static behavior
• Output-prediction MCQs

Execution Count of Static Static vs Non-Static Members in Java

① STATIC MEMBERS – How many times do they execute?



Runs Once Per Class During Class Loading

Static Item	What It is Used For	When It Executes	Trigger
Static Variable (Initialization)	Shared class-level data	During class initialization	Class. Appliance – dj.–
Static Final Variable	Constants	During class	Calsty Jnalliezatioe
Static Block	Complex, Initialization logic	During class Initialization	DB ottver coming kevet
Static Method	Utility, Inelper logx.	Only when called	Maiin-Urtic, Callengions
Static Factory Method	Centrolled object creation	When called	3ethistanc(O
Static Main Method	JVM entry point	After static int	Class

```
class StaticExample {
  static int a = initA(); // static variable
}
  static {; // static block
    System.out.printin("Static block excu)
}
  static int initA() { // static mnethod
    System.out.printin("Static variable init-
}
```

Output Order:

- Static variable initialized
- Static black executed
- Main started

② NON-STATIC MEMBERS – How many times do they execute?

Runs **Every Time** a New Object is Created

Scenario	Static Members	Trigger
Single object	Once per object	Class / Abstriple cods
Multiple objects	Once per object	Class / Apaliple onds
Inheritance	Once per object	Class / Abstriple cods
Interface	Once per object	Class / Apaliple cods
Abstract class	Every time it's called	Object
Multiple JVM runs	Once per threatobjet	Chlirid object Cheram-poice
Multiple ClassLoaders	Once per object	Thread class Interface

EXECUTION COUNT BY SCENARIO (MASTER VIEW)

Scenario	Static Members	Non-Static
Single object	1 time	✔
Inheritance	Parent- class loading	Per object
Interface	Stack	✔
Methettreading	1	Per' VMseerselee
Multiple JVM	Per object	Per object
Multiple ClassLoaders	1 Teriper rum	Per object

JVM Memory Corretion (Code Dve)	
Single object	1 time
Multiple objects	1 time
Inheritance	Per Object

Member Type	Method Area /	Non-Static Members
Static	Method Area /	Once
Non-Static	Heap	Per object
Thread execution	Stack	Per thread

Static members execute once per class per • ClassLoader; non-static members execute once per object

Below is a **complete, precise, and interview-safe table** that answers **exactly "how many times"** each **static** and **non-static** member executes **across ALL Java scenarios** (normal class, inheritance, interfaces, multithreading, JVM lifetime).

🔥 **Execution Count of Static vs Non-Static Members in Java**

📊 **STATIC MEMBERS –** ***How many times do they execute?***

Rule: Static members belong to the **class**, not objects.

Static Member	How Many Times It Executes	Trigger	Applies To	Important Notes
Static Variable (Initialization)	**ONCE per class per ClassLoader**	Class loading	Class / Abstract class	Not per object
Static Final Variable (Constant)	**ONCE**	Compile / class init	Class / Interface	Inlined at compile time
Static Block	**ONCE per class per ClassLoader**	Class initialization	Class / Abstract class	Runs before main()
Static Method	**ONLY when called**	Method invocation	Class	Can be called many times
main() Method	**ONCE per JVM run**	JVM entry	Class	Static method
Static in Parent Class	**ONCE**	Parent class loading	Inheritance	Before child
Static in Interface	**ONCE**	First reference	Interface	No instance lifecycle
Static in Multithreading	**ONCE**	Class loading	All threads	Shared across threads
Static in Multiple Objects	**ONCE**	Class loading	Many objects	Not repeated

📌 **Key Rule:**

Static initialization never repeats unless **ClassLoader changes**.

📊 **NON-STATIC MEMBERS – *How many times do they execute?***

Rule: Non-static members belong to the **object**.

Non-Static Member	How Many Times It Executes	Trigger	Applies To	Important Notes
Instance Variable	**Once per object**	Object creation	Class / Abstract class	Defaults → explicit
Instance Block	**Once per object**	Object creation	Class / Abstract class	Before constructor
Constructor	**Once per object**	Object creation	Class / Abstract class	Mandatory
Non-Static Method	**Every time it's called**	Method invocation	Object	Can be many times
Instance in Parent Class	**Once per object**	Child object creation	Inheritance	Parent → child
Instance in Child Class	**Once per object**	Child object creation	Inheritance	After parent
Non-Static in Multithreading	**Once per thread object**	new Thread()	Thread class	Per thread
Non-Static in Interface	✕ **Never**	✕	Interface	No instance state

---

🔁 **EXECUTION COUNT BY SCENARIO (MASTER VIEW)**

Scenario	Static Members	Non-Static Members
Single object	1 time	1 time
Multiple objects	1 time	Per object

Inheritance	Parent → 1, Child → 1	Parent → per object, Child → per object
Interface	Static → 1	✕ Not allowed
Abstract class	1	Per subclass object
Multithreading	1 (shared)	Per thread object
Multiple JVM runs	1 per run	Per object per run
Multiple ClassLoaders	1 per loader	Per object

🧠 JVM MEMORY CORRELATION (INTERVIEW GOLD)

Member Type	Memory Area	Execution Frequency
Static	Method Area / Metaspace	Once
Non-Static	Heap	Per object
Thread execution	Stack	Per thread

🎯 PERFECT INTERVIEW ONE-LINER

"Static variables and blocks execute once per class per ClassLoader, while non-static variables, blocks, and constructors execute once per object; methods execute every time they are called."

If you want next 🔥

- Output-based MCQs
- Tricky interview rejection cases
- Static + inheritance corner cases
- ClassLoader deep dive